

Phase 1: Sorting Foundations and Evaluation Criteria

1. Main Goal of Phase 1

Phase 1 is the foundation for learning sorting algorithms.

In this phase, we are **not learning a full sorting algorithm yet**. Instead, we are learning the important words and ideas used to understand and compare sorting algorithms.

Before learning Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and other sorting techniques, you must understand:

- What sorting means
- Why sorting is useful
- How we judge sorting algorithms
- What comparisons, swaps, shifts, adaptiveness, memory, and stability mean
- What Big O notation means
- Why swapping in C uses pointers

2. What Is Sorting?

Sorting means arranging data in a meaningful order.

Most of the time, we sort data in one of two ways:

1. **Ascending order**
2. **Descending order**

3. Ascending Order

Ascending order means arranging values from smallest to largest.

Example:

```
Before sorting:  
8 3 7 2 5
```

```
After sorting in ascending order:  
2 3 5 7 8
```

Here, the smallest value comes first and the largest value comes last.

4. Descending Order

Descending order means arranging values from largest to smallest.

Example:

```
Before sorting:  
8 3 7 2 5  
  
After sorting in descending order:  
8 7 5 3 2
```

Here, the largest value comes first and the smallest value comes last.

5. Why Sorting Is Important

Sorting is useful because it makes data easier to:

- Search
- Compare
- Display
- Organize
- Rank
- Analyze

Examples:

- A school may sort students by marks.
- A shop may sort products by price.
- A database may sort records by name.
- A website may sort search results by relevance.
- A leaderboard may sort players by score.

Sorting is one of the most common operations in computer science.

6. The Beginner Mistake

A beginner may ask only this question:

Does this algorithm sort the array?

But that question is too simple.

A better question is:

How efficiently does it sort, how much memory does it use, and does it preserve important order?

Many sorting algorithms can sort data, but they do it in different ways.

Some algorithms are:

- Easy but slow
- Fast but complicated
- Good for small data
- Good for large data
- Good when the data is already almost sorted
- Good when memory is limited
- Stable with duplicate values
- Not stable with duplicate values

So, we need criteria to judge sorting algorithms properly.

7. Five Main Criteria for Judging Sorting Algorithms

We judge sorting algorithms using five important criteria:

1. Number of comparisons
2. Number of swaps or shifts
3. Adaptiveness
4. Extra memory
5. Stability

These criteria help us decide whether one sorting algorithm is better than another for a particular situation.

8. Criterion 1: Number of Comparisons

Meaning

A **comparison** happens when an algorithm checks two values to decide their order.

Example:

```
if (A[j] > A[j + 1])
```

This means:

Is the value at A[j] greater than the value at A[j + 1]?

Simple Example

Suppose we have this array:

```
8 3 7
```

The algorithm may compare:

```
8 > 3
```

This is one comparison.

If the algorithm compares many values again and again, the sorting process becomes slower.

Why Comparisons Matter

Comparisons are important because many sorting algorithms make decisions by comparing values.

If an algorithm makes too many comparisons, it can become slow for large arrays.

For example:

- Sorting 5 numbers may be quick.
- Sorting 5,000,000 numbers may become very slow if there are too many comparisons.

So, fewer comparisons usually means better performance.

9. Criterion 2: Number of Swaps or Shifts

Sorting algorithms do not only compare values. They also move values.

There are two common ways values are moved:

1. Swapping
 2. Shifting
-

9.1 What Is a Swap?

A **swap** means two values exchange places.

Example:

Before swap:

8 3

After swap:

3 8

The value **8** moved to the right, and the value **3** moved to the left.

9.2 What Is a Shift?

A **shift** means moving values one position to make space for another value.

Example:

2 6 10 15

Suppose we want to insert **7** into the correct position.

First, we shift **10** and **15** to the right:

2 6 _ 10 15

Then we put **7** into the empty space:

2 6 7 10 15

9.3 Why Swaps and Shifts Matter

Moving data takes time.

In some systems, writing to memory is expensive. So, an algorithm that moves values fewer times may be better.

Some algorithms use many swaps. Some algorithms use many shifts. Some algorithms try to reduce movement.

For example:

- Bubble Sort may do many swaps.
- Insertion Sort often uses shifts.
- Selection Sort does fewer swaps than Bubble Sort.

10. Criterion 3: Adaptiveness

Meaning

An algorithm is **adaptive** if it becomes faster when the data is already sorted or nearly sorted.

Simple Example

Suppose the array is already sorted:

2 3 5 7 8

A smart sorting algorithm should notice this and stop early or do very little work.

An adaptive algorithm behaves like this:

This data is already sorted, so I do not need to do much work.

A non-adaptive algorithm behaves like this:

I will keep checking everything even though the data is already sorted.

Why Adaptiveness Matters

Real-world data is often not completely random.

Sometimes data is already:

- Fully sorted
- Almost sorted
- Partly sorted

In such cases, adaptive algorithms can save a lot of time.

Example:

If a list is already sorted, an adaptive version of Bubble Sort can stop early.

11. Criterion 4: Extra Memory

Meaning

Extra memory means additional space used by an algorithm while sorting.

Some algorithms sort data inside the original array. Others need extra arrays, nodes, or recursion stack space.

11.1 In-Place Sorting

An algorithm is called **in-place** if it sorts mostly inside the original array and does not need a large extra array.

Example:

```
Original array:  
8 3 7 2
```

```
Same array after sorting:  
2 3 7 8
```

The sorting happens inside the same array.

11.2 Algorithms That Need Extra Memory

Some algorithms need extra memory.

For example:

- Merge Sort uses an extra array for merging.
 - Count Sort uses a counting array.
 - Bin Sort uses bins or linked lists.
 - Recursive algorithms use the recursion stack.
-

Why Extra Memory Matters

Memory is limited.

If an algorithm is fast but uses too much memory, it may not be suitable for some systems.

For example:

Count Sort can be fast, but if the largest value is very large, it may need a huge counting array.

So, we should not judge an algorithm only by speed. We should also check how much memory it uses.

12. Criterion 5: Stability

Meaning

A sorting algorithm is **stable** if it keeps equal values in their original relative order.

This idea is very important when sorting records.

Simple Stability Example

Suppose we have students with marks:

C: 6 marks
E: 6 marks

Both students have the same marks.

If **C** appears before **E** before sorting, a stable sorting algorithm keeps **C** before **E** after sorting.

Full Example

Before sorting by marks:

A: 9
C: 6
D: 8
E: 6

After stable sorting by marks:

C: 6
E: 6
D: 8
A: 9

Notice that **C** still comes before **E**.

That is stability.

What Stability Does Not Mean

Stability does not mean the array is already sorted.

Stability only means:

Equal values keep their original order.

Why Stability Matters

Stability is useful in database-style sorting.

Example:

Suppose we sort students by name first.

Then we sort them by marks.

If the second sort is stable, students with the same marks will still remain in name order.

This is useful when sorting by multiple fields.

13. Comparison-Based Sorting

Sorting algorithms can be grouped into two major types:

1. Comparison-based sorting
 2. Index-based sorting
-

Meaning of Comparison-Based Sorting

A **comparison-based sorting algorithm** sorts values by directly comparing them.

It asks questions like:

Is $A[i]$ greater than $A[j]$?
Is $A[i]$ smaller than the pivot?
Is $A[j]$ greater than $A[j + 1]$?

Examples of Comparison-Based Sorting Algorithms

Examples include:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Quick Sort

- Merge Sort
- Shell Sort

Most beginner sorting algorithms are comparison-based.

Simple Comparison-Based Example

Suppose we have:

8 3

The algorithm compares:

8 > 3

Since 8 is greater than 3, it may swap them.

Result:

3 8

14. Index-Based Sorting

Meaning of Index-Based Sorting

An **index-based sorting algorithm** does not mainly sort by comparing pairs of values.

Instead, it uses the value itself as:

- An index
 - A position
 - A bucket
 - A digit
-

Examples of Index-Based Sorting Algorithms

Examples include:

- Count Sort
 - Bin Sort
 - Radix Sort
-

Simple Count Sort Idea

Suppose the value `6` appears in the array.

Count Sort may do this:

```
counter[6]++
```

This means:

Increase the count of value 6.

Instead of comparing `6` with many other values, the algorithm uses `6` as an index.

Advantage of Index-Based Sorting

Index-based sorting can be very fast when conditions are suitable.

For example, Count Sort can be very fast when the range of numbers is small.

Disadvantage of Index-Based Sorting

Index-based sorting may use a lot of extra memory.

Example:

If we want to sort:

```
2 10000
```

Count Sort may need a counter array from index `0` to index `10000`.

That is a lot of memory for only two values.

15. Big O Notation

Meaning

Big O notation describes how the work of an algorithm grows as the input size grows.

It does not tell the exact number of seconds.

Instead, it tells the growth pattern.

What Is n ?

In Big O notation, n usually means the number of items in the input.

Example:

If an array has 10 elements, then:

$n = 10$

If an array has 1,000 elements, then:

$n = 1000$

16. $O(1)$: Constant Time

Meaning

$O(1)$ means constant time.

The amount of work does not grow with the size of the array.

Example

Swapping two selected numbers is $O(1)$.

Whether the array has 5 items or 5 million items, swapping two selected values still takes only a few steps.

Example:

```
int temp = *x;  
*x = *y;  
*y = temp;
```

This always performs a fixed number of operations.

17. $O(n)$: Linear Time

Meaning

$O(n)$ means linear time.

The work grows directly with the number of elements.

Example

```
for (int i = 0; i < n; i++) {  
    printf("%d ", A[i]);  
}
```

This loop visits each element once.

If $n = 10$, it does about 10 steps.

If $n = 1000$, it does about 1000 steps.

Beginner Meaning

$O(n)$ means:

If the input doubles, the work roughly doubles.

18. $O(n^2)$: Quadratic Time

Meaning

$O(n^2)$ means quadratic time.

This often happens when there are nested loops.

Example

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        // work  
    }  
}
```

The inner loop runs many times for each run of the outer loop.

Simple Growth Example

If $n = 10$, the work may be around:

$10 \times 10 = 100$ operations

If $n = 100$, the work may be around:

$100 \times 100 = 10,000$ operations

So, $O(n^2)$ grows very quickly.

Sorting Algorithms Often Related to $O(n^2)$

Examples:

- Bubble Sort
- Selection Sort
- Insertion Sort in the worst case
- Quick Sort in the worst case

These algorithms can become slow for large input if they behave like $O(n^2)$.

19. $O(n \log n)$

Meaning

$O(n \log n)$ is usually much better than $O(n^2)$ for large data.

You do not need to fully understand logarithms yet.

For now, remember:

$O(n \log n)$ is usually a fast class for comparison-based sorting.

Examples

Algorithms that often have $O(n \log n)$ behavior include:

- Merge Sort
 - Quick Sort in the best and average cases
 - Heap Sort
 - Tree Sort
-

Beginner Meaning

For large arrays:

$O(n \log n)$ is usually much better than $O(n^2)$

That is why algorithms like Merge Sort and Quick Sort are important.

20. Other Big O Categories Mentioned

The curriculum also mentions other time categories.

$O(n^{3/2})$ or Nearby

This is connected with Shell Sort in some analyses.

Shell Sort can often be faster than $O(n^2)$, but its exact performance depends on the gap sequence used.

$O(d \times n)$

This is connected with Radix Sort.

Here:

- n means number of elements
- d means number of digits in the largest number

Radix Sort may process numbers digit by digit.

21. Memory Behavior in C

In C, memory behavior is very important.

When you pass a normal variable to a function, the function usually receives a copy.

If the function changes the copy, the original variable does not change.

To change the original variable, we pass its address.

This is why swapping in C needs pointers.

22. Why Swapping in C Needs Pointers

Suppose we have two variables:

```
int a = 10;  
int b = 20;
```

We want a function to swap their values.

If we call a function with only the values, the function may swap local copies, not the real variables.

To swap the real variables, we pass their addresses:

```
swap(&a, &b);
```

The `&` symbol means:

address of

So:

```
&a
```

means:

the memory address of variable a

23. Pointer-Based Swap Function

Here is the swap function:

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

Meaning of `int *x`

```
int *x
```

means:

x is a pointer to an integer

A pointer stores an address.

So, `x` stores the address of one integer, and `y` stores the address of another integer.

Meaning of `*x`

`*x`

means:

the value stored at the address inside `x`

If `x` stores the address of `a`, then `*x` means the value of `a`.

24. Dry Run of the Swap Function

Suppose:

```
int a = 10;  
int b = 20;
```

We call:

```
swap(&a, &b);
```

Before the swap:

```
a = 10  
b = 20
```

Step 1

```
int temp = *x;
```

`*x` is the value of `a`, so:

```
temp = 10
```

Now:

```
a = 10  
b = 20  
temp = 10
```

Step 2

```
*x = *y;
```

*y is the value of b, which is 20.

So the value of b goes into a.

Now:

```
a = 20  
b = 20  
temp = 10
```

Step 3

```
*y = temp;
```

The saved value 10 goes into b.

Now:

```
a = 20  
b = 10
```

The swap is complete.

25. Why We Need `temp` in Swap

The variable `temp` protects one value while the other value overwrites it.

If we do not use `temp`, one value may be lost.

Example:

```
*x = *y;  
*y = *x;
```

This is wrong because after the first line, both values may become the same.

Correct method:

```
int temp = *x;  
*x = *y;  
*y = temp;
```

26. Display Function

A display function is used to print the array.

Example:

```
void display(int A[], int n) {  
    for (int i = 0; i < n; i++) {  
        printf("%d ", A[i]);  
    }  
    printf("\n");  
}
```

Meaning

- `A[]` is the array.
- `n` is the number of elements.
- The loop prints each element from index `0` to index `n - 1`.

This function will be useful in later sorting programs.

27. Complete Phase 1 Program

```
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void display(int A[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

int main() {
    int a = 10, b = 20;

    printf("Before swap: a=%d, b=%d\n", a, b);

    swap(&a, &b);

    printf("After swap: a=%d, b=%d\n", a, b);

    return 0;
}
```

28. Expected Output

```
Before swap: a=10, b=20
After swap: a=20, b=10
```

29. Explanation of the Program

The program has three main parts:

1. `swap()` function
 2. `display()` function
 3. `main()` function
-

29.1 `swap()` Function

The `swap()` function receives the addresses of two integers.

It swaps the real values using pointers.

```
void swap(int *x, int *y)
```

This means the function accepts two integer pointers.

29.2 `display()` Function

The `display()` function prints all elements of an array.

It uses a loop from `0` to `n - 1`.

29.3 `main()` Function

The `main()` function creates two variables:

```
int a = 10, b = 20;
```

It prints their values before swapping.

Then it calls:

```
swap(&a, &b);
```

Then it prints their values after swapping.

30. Complexity of Swap

The swap function is $O(1)$ time.

Reason:

It always performs a fixed number of steps:

```
int temp = *x;  
*x = *y;  
*y = temp;
```

It does not matter how large the array is.

Space Complexity of Swap

The swap function uses $O(1)$ extra space.

Reason:

It only uses one extra variable:

```
temp
```

So both time and space complexity of swap are constant.

31. Common Beginner Mistakes

Mistake 1: Confusing Stability with Sortedness

Stability does not mean the array is sorted.

Stability means:

If two values are equal, their original order is preserved.

Mistake 2: Thinking $O(n)$ Is Always the Best

An $O(n)$ algorithm can still be unsuitable if it uses too much memory.

Example:

Count Sort may be fast, but if the largest value is huge, it may need a huge counting array.

So, speed is not the only thing to consider.

Mistake 3: Forgetting `&` in Swap

Wrong:

```
swap(a, b);
```

Correct:

```
swap(&a, &b);
```

The function needs the addresses of `a` and `b` so it can modify the real variables.

Mistake 4: Forgetting What `*` Means

In a pointer function:

```
*x
```

means:

the value at the address stored in x

Beginners often confuse the pointer itself with the value it points to.

Mistake 5: Judging Algorithms Only by Speed

Speed is important, but it is not the only factor.

You should also ask:

- Is the algorithm stable?
- Is it adaptive?
- Does it use extra memory?
- Does it make many swaps?
- Is it good for this type of data?
- Is the input already sorted or nearly sorted?
- Are there many duplicate values?

32. Key Vocabulary Table

Term	Simple Meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already or nearly sorted
Non-adaptive	Does not become much faster on already sorted data
Extra memory	Additional space used by an algorithm
In-place	Sorts mainly inside the original array
Stable	Keeps equal values in their original relative order
Comparison-based sort	Sorts by comparing values directly
Index-based sort	Uses values as indices, buckets, or digits
Big O	Describes how work grows as input grows
$O(1)$	Constant time
$O(n)$	Linear time
$O(n^2)$	Quadratic time
$O(n \log n)$	Fast comparison-sorting class
Pointer	A variable that stores an address

Term	Simple Meaning
<code>&</code>	Address of
<code>*</code>	Value at an address
<code>temp</code>	Temporary variable used to protect a value during swap

33. Sorting Algorithm Evaluation Checklist

When you study any sorting algorithm, ask these questions:

Comparisons

- How many comparisons does it make?
- Does it compare neighboring elements or distant elements?

Movement

- Does it use swaps?
- Does it use shifts?
- Does it move data many times?

Adaptiveness

- Is it faster when the array is already sorted?
- Can it stop early?

Memory

- Does it sort in-place?
- Does it need an extra array?
- Does it use recursion stack space?
- Does it use linked lists or bins?

Stability

- Does it keep duplicate values in their original order?
- Is stability important for the problem?

Time Complexity

- What is the best-case time?
- What is the average-case time?

- What is the worst-case time?

Practical Suitability

- Is it good for small data?
 - Is it good for large data?
 - Is it good for nearly sorted data?
 - Is it good when memory is limited?
-

34. Phase 1 Summary

Phase 1 teaches the foundation needed before learning sorting algorithms.

The most important idea is this:

Do not judge a sorting algorithm only by whether it sorts. Judge it by how it sorts.

You learned that sorting algorithms can be judged by:

1. Number of comparisons
2. Number of swaps or shifts
3. Adaptiveness
4. Extra memory
5. Stability

You also learned:

- Sorting means arranging data in order.
 - Ascending means smallest to largest.
 - Descending means largest to smallest.
 - Comparison-based sorts compare values directly.
 - Index-based sorts use values as indices, buckets, or digits.
 - Big O describes growth of work as input size increases.
 - $O(1)$ means constant time.
 - $O(n)$ means linear time.
 - $O(n^2)$ means quadratic time.
 - $O(n \log n)$ is usually a fast class for comparison sorting.
 - C swap functions need pointers to modify real variables.
 - $\&$ means address of.
 - $*$ means value at an address.
-

35. What Comes Next

After Phase 1, you are ready for **Phase 2: Bubble Sort**.

Bubble Sort will use many Phase 1 ideas, especially:

- Comparisons
- Swaps
- Passes
- Adaptiveness
- Stability
- Big O

Phase 1 gives you the vocabulary. Phase 2 starts applying it to a real sorting algorithm.